

TrainVerify 技术报告提纲

从分布式训练图变换到内核检查的正确性证明

提纲草案，面向 OSDI/SOSP 风格系统论文

2026 年 8 月 11 日

摘要

这不是论文正文，而是一份写作与证据规划。目标读者懂机器学习或系统研究，但不要求预先了解分布式训练、nnScaler、YOCO 或 Lean。报告首先解释训练代码如何变成单设备图和分布式图，再说明这种变换为什么涉及值的所有权、通信和布局，因而不能靠形状检查保证正确。随后介绍 TrainVerify 如何把真实图翻译为 Lean 语义、构造完整命题、组合局部证明并发布可审计结果。最后把系统的可复用部分、尚未泛化的工程和不可避免模型特化工作分开。

论文主张与写作原则

拟建立的核心论点：分布式图编译的正确性不是“两个图能运行且 shape 相同”，而是分布式图中每个设备的局部值能否按照真实通信和布局规则重构为单设备图的值。TrainVerify 将这一关系编译成 Lean 可检查的命题，并在真实 Transformer/MoE/context-parallel 图上闭合了五个完整观测测量。

- 论文必须同时解释被验证的系统和验证系统，不能从 Lean 文件结构起笔。
- 所有图、术语和例子先给直觉，再给代码或形式定义。
- “Lean 定理成立”“定理绑定真实 GPU 图”“工件完成 sealed publication”是三层证据，全文保持分离。
- 当前 YOCO 结果证明可行性和规模，不足以声称已经实现适用于任意模型的一键 proof compiler。
- 摘要、引言、贡献列表、评估和结论最终必须陈述同一个强度的结果。

1 引言

本节任务：让完全不懂分布式训练的系统研究者在两页内理解问题、失败后果、TrainVerify 的方法和本文贡献。

1.1 一个形状正确但值错误的例子

从长度为 8 的序列出发。展示普通连续切分与 zigzag 切分得到相同局部 shape，但 rank-order concat 只能还原前者。说明通信程序可以成功、shape 可以一致，最终值仍然排列错误。

1.2 问题陈述

训练框架和图编译器把单设备程序变成每个 rank 的局部程序，并插入分片、复制、AllGather、AllToAll、shuffle 和 adapter。正确性目标是：对明确的外部输入条件，PM 的局部执行结果按声明关系重构后与 SM 结果一致。

1.3 TrainVerify 的方法

用一段连续叙事概述：固定训练代码与编译器 revision；从真实双 GPU 运行提取 SM/PM authority；翻译节点、张量和通信语义；沿 backward ancestry 生成命题；用局部 relation rule 组成 proof DAG；交给 Lean 检查；用 registry 与 content-addressed snapshot 绑定源码和结果。

1.4 贡献列表

最终正文只保留有评估证据支撑的贡献。候选贡献如下：

1. 一个面向分布式训练图的值语义与所有权关系模型，区分连续分片、zigzag、复制、expert partition 和 hidden sharding。
2. 一个 authority-to-Lean pipeline，将真实 SM/PM 图、来源记录和输入条件翻译为可检查命题。
3. 一套从局部算子关系到完整 backward ancestry 的证明组合方法，并在 YOCO-MoE-A0.4B 的五个观测量上形成完整定理。
4. 一套 fail-closed 诊断与发布机制，区分不支持的算子、缺失关系、错误命题、证书错误和信任链错误。
5. 对系统泛化边界的实证分析，明确哪些机制已通用、哪些只是应当通用、哪些仍需模型或框架专家参与。

图表计划：图 1：从 llm-train 源码到 nnScaler SM/PM 图，再到 TrainVerify 命题、证明和 sealed result 的整条链。该图应成为全文导航图。

2 背景：训练代码如何变成 SM 和 PM

本节任务：不假设读者了解 YOCO、nnScaler 或分布式并行。讲清楚“被转换的对象是什么”和“谁负责哪一步”。

2.1 从一次训练迭代开始

按数据流解释一轮训练中与本文有关的组件：输入 token 与 metadata、embedding、decoder layer、attention、MoE router 与 experts、loss、反向传播。说明图节点不仅包含模型层，也可能包含 adapter、collective、stack、reshape 和 autograd 节点。

2.2 SM 图里有什么

定义 SM 为单设备参考图，而不是“只做 forward 的简化模型”。正文应配一张缩略图，至少标出：

- 外部输入、参数和初始化 store;
- 普通算子节点，如 embedding、linear、RMSNorm、attention、MoE、loss;
- tensor ID、shape、producer-consumer edge;
- 本文选择的五个 observable;
- 与训练代码位置和源码 revision 的对应关系。

2.3 PM 图里新增了什么

定义 PM 为经过并行规划后的图。按两个 rank 展示：

- rank-local operator 与 tensor shard;
- replicated 参数、expert-local 参数和 buddy relation;
- AllReduce、AllGather、ReduceScatter、AllToAll;
- nnScaler 自动插入的 adapter;
- process group、topology、partition annotation 和 profile;
- 同一个 SM 节点在 PM 中可能展开成多个节点或不同拓扑。

2.4 llm-train 的职责

从源码与 API 层说明：llm-train 定义 YOCO 模型、算子调用、custom op、并行参数和训练逻辑。需要逐项核对哪些 annotation 或 wrapper 会影响 nnScaler trace 和 planner，避免把模型代码、编译器插入和 runtime kernel 混称为同一层。

2.5 nnScaler 的职责

按真实 pipeline 解释：llm-train compile API 提供 module、dummy inputs 和并行配置；nnScaler trace 得到 IRGraph 并补出 backward；PAS/AutoDist 读取约束与通信、算子 profile，决定节点的复制、切分和放置；IRAdapterGener 在不匹配的布局之间插入 Chunk、AllGather、AllReduce 或 AllToAll；ExecutionPlan 和 ModuleCodeGen 最后生成各 rank 程序及 graph pickle。正文要以 nnscale/parallel.py 的实际 _gencode 路径和源码位置核对这条时序，不能只画概念框。

2.6 为什么 SM 到 PM 的转换不平凡

用四个递进例子说明：

1. 连续 tensor shard：形状与 concat 维度必须正确；
2. context parallel zigzag：局部 shape 不携带排列信息；
3. MoE：token routing、expert ownership 和 AllToAll 相互依赖；
4. mixed-layout attention：Q 与 decoder stream 在 shuffle 后为 zigzag，K/V 却来自 shuffle 前的普通连续 shard。

拟建立的核心论点：图变换的关键状态不仅是 shape，还包括值来源、rank ownership、排列、复制组和 collective scope。

图表计划：图 2：一个 YOCO layer 的 SM/PM 对照图；图 3：ordinary 与 zigzag 两种相同 shape 的布局；图 4：llm-train、nnScaler compiler、runtime 三者的调用时序。

3 问题定义与威胁模型

本节任务：把“正确”写成可验证的系统属性，并说明本文防什么、不防什么。

3.1 输入、输出和量化范围

定义固定 revision 的 SM graph、PM graph、authority metadata、外部输入 store、rank-local store、目标 observable 和关系。给出自然语言主定理模板，再给一个简化形式公式。

3.2 错误模型

覆盖模型 API 误用、custom-op annotation 错误、planner/adaptor 错误、runtime selector 错误、TrainVerify 翻译错误、命题弱化、stale artifact 和 provenance mismatch。区分值错误、表达能力不足、实现不支持和发布链错误。

3.3 非目标

明确不证明 CUDA bitwise equality、完整训练收敛、模型质量、任意硬件一致性，也不把五个 observable 外推成完整训练程序等价。

4 TrainVerify 总体设计

本节任务：先讲模块职责和信息流，再进入 Lean 细节。

4.1 Authority capture

解释为什么必须从真实双 GPU、双 rank NCCL production path 取得图；CPU smoke 与 planner-only run 能检查什么、不能检查什么；revision、profile、hardware 和 artifact digest 如何进入来源记录。

4.2 Graph extraction and normalization

说明如何读取 nnScaler 图，恢复有序节点、输入输出、operator 参数、rank、replica group、shape 和 producer relation。列出拒绝不完整或歧义图的条件。

4.3 从 nnScaler IR 到 verifier DFG

单独解释 Verdict backend 不是机械 pickle dump。它需要展开 segment 与 adaptor、按 rank 展开节点、规范化 operator、补齐 backward 所需的 forward inputs、处理常量、强制单一赋值、注入 gradient accumulation，并恢复 shape/value map 与 logical replica identity。这里是第二个需要

人工 fidelity audit 的边界；input order、reshape 参数、broadcast、fused collective、writer/version 和 rank subgroup 都可能改变语义。

4.4 Graph-to-Lean translation

概述生成 GraphDecl、节点、shape environment、init goals、input value classes、observables 和 manifest。将翻译器视为不可信 producer，其输出需要 Lean 与独立 fidelity audit 检查。

4.5 Denotational semantics

定义 store 与 graph fold。分别介绍普通 evaluator 和 faithful distributed evaluator，解释后者为什么需要看到另一 rank、replica-buddy、process group 和 permutation。

4.6 Relation and proof layer

介绍 contiguous shard、zigzag、replicated、expert partition、hidden shard 和 partial reduction relation。说明局部 operator theorem 如何转换 relation，并组成 graph-level certificate。

4.7 Kernel check and publication

说明 theorem type、axiom footprint、non-vacuity witness、all-source direct build、proof registry 与 final receipt 的角色。供应链加固细节放 artifact appendix，不挤占设计主线。

图表计划：图 5: TrainVerify 内部架构，标出可信边界；图 6: authority graph 到 Lean declarations、obligations、certificate DAG 的数据结构变化。

5 翻译层：需要人类审查什么

本节任务：把“翻译正确性”从一句 disclaimer 变成可操作的审查协议。

5.1 组件映射表

建立一张长期维护的 paper-to-source map：

上游对象	Lean 对象	自动检查	人工检查
模型或 custom op	operator denotation	signature、shape、fixture	值语义与 Python 源码一致
rank-local tensor	relation instance	producer、rank、shape	ownership 与排列来源
collective	transition theorem	group 与维度	实际 runtime 语义
目标切片	goal graph	backward closure	未漏 graph-computed input
输入条件	theorem premise	类型与 witness	条件不偷渡结论

5.2 算子语义审查

对每类 operator 固定审查模板：Python 实现、input/output contract、shape、值公式、边界分支、world size、process-group scope、Lean declaration、最小例子和 mutation test。

5.3 布局与所有权审查

禁止从 shape、tensor 名或 TID 猜 ownership。要求沿 producer edge、collective 和 annotation 还原 ordinary、zigzag、replicated 或 expert-local provenance。

5.4 翻译验证策略

规划三类证据：小张量可执行对照；结构和 digest 对照；mutation test。明确这些证据提高 fidelity 信心，但不等价于对 Python-to-Lean translator 本身做完形式化证明。

图表计划：表 1: operator coverage 与人工审查状态；表 2: relation coverage；图 7: mixed Q/K/V translation audit 示例。

6 命题如何生成

本节任务：解释为什么一个“能证明”的命题仍可能是错的，以及 TrainVerify 如何构造不偷懒的目标。

6.1 从 observable 反向闭合

从目标 TID 同时沿 SM 和 PM producer graph 回溯，直到只剩真实外部输入。解释 backward ancestry closure 如何避免把 graph-computed tensor 暴露成自由 premise。

6.2 输入条件

区分 shape environment、init goals、input value classes、packed sequence metadata 和 label bounds。每个条件必须在执行前独立检查，并尽可能提供 satisfiability witness。

6.3 中间 obligation

说明何时需要 shape fact、producer fact、relation fact、scope bridge 和 local semantic side condition。false or unsupported obligation 必须保留为结果，不能缩小 denominator。

6.4 公开 theorem

说明 full、cut 和 legacy theorem 的命名隔离；公开 theorem 必须直接指向 full statement；Instances 与 MainTheorem 如何形成最终接口。

7 命题如何证明

本节任务：给出读者可复述的证明路线，而不是文件名和 TID 清单。

7.1 局部规则库

把规则写成“输入 relation + operator semantics + side conditions 推出输出 relation”。按 elementwise、linear、reshape、attention、MoE、collective、shuffle/unshuffle 分类。

7.2 图级组合

说明如何按 producer 顺序维护 relation environment；如何选择规则、生成有限 graph certificate、处理 graph drift 和 scoped bridge；为什么 Graph A 的事实不能无桥搬到 Graph B。

7.3 YOCO 五个证明

用一张共享依赖图解释：Goals 1/2 共享长 producer prefix 但有不同 loss tail；Goals 3/4 在各自 observable scope 内重构 stack；Goal 5 闭合 hidden shard 与 AllToAll 小图。把 58-file checkpoint 等工程细节放附录。

7.4 防止 vacuous success

覆盖 theorem-header audit、`#print axioms`、input-contract witness、禁止 computed premise、禁止 identity shuffle 和 stale olean。

图表计划：图 8：从 external input relation 到 ordinary prefix、shuffle、zigzag stream、mixed attention、unshuffle/gather 和 observable 的证明 DAG。

8 实现

本节任务：给足可复现信息，但不重复设计。

包括 authority scripts、graph emitter、Lean semantic library、rule modules、registry、clean-room direct builder 和 receipt。报告代码规模、生成与手写比例、构建资源和运行时间时，只使用最终可复查日志。不要把易变的 TID 或临时目录当作论文贡献。

9 评估

本节任务：用研究问题组织实验，证明系统有用、正确、可扩展，而不只是“YOCO build 绿了”。

9.1 RQ1：翻译是否忠实

对代表性 operator、collective 和布局做源级审计、小张量对照与 mutation test。报告发现过的 verifier-side translation bugs，以及每类 mutation 是否在预期阶段被拒绝。

9.2 RQ2：能否处理真实复杂图

报告 SM/PM graph 规模、operator 类别、collective、层数、五个 observable、proof DAG 规模、Lean build 时间和峰值内存。按“faithful denotation、value-lossy legacy、unsupported”分类原始节点和 obligation，解释为何选择这些 observables，以及它们覆盖哪些困难路径。五个公开 full

theorem 不等于原始 obligation 全部为真；两个 ordinary-gather false findings 必须继续计入分母并单独报告。

9.3 RQ3: 比传统检查多发现什么

与 shape checking、单次 numerical differential test、CPU smoke 和 compiler self-check 对比。案例包括 CC12 selector failure、nnScaler zigzag/RVD gap、TrainVerify 自身 denotation bugs，以及一次被源码证伪并撤回的 K/V 错误归因。

9.4 RQ4: 多少工作已经自动化

分别报告自动生成、通用库复用、模板化生成、YOCO-specific handwritten proof 和人工 fidelity audit。禁止用单一 coverage 百分比掩盖手写工作。

9.5 RQ5: 失败诊断是否有用

构造 unsupported operator、missing relation、ambiguous ownership、false goal、certificate corruption 和 provenance mismatch，检查系统能否定位第一个失败节点、tensor、候选规则和原因。

9.6 RQ6: 泛化到新模型的成本

选择至少一个未用于开发规则库的 architecture 做 clean-room test。成功时报告新增 adapter、denotation、relation rule 和 proof code；失败时按结构化 failure contract 报告。没有 unseen-architecture 结果前，不声称已实现通用 proof compiler。

9.7 RQ7: 信任链能否拒绝篡改

分别注入 stale .olean、statement drift、registry corruption、unexpected axiom、不可满足输入条件和 provenance mismatch，验证它们在预期门禁被拒绝。把 sealed provenance 与 semantic fidelity 分开评价。

9.8 RQ8: 实际诊断质量

衡量系统是否定位到第一个失败节点和 tensor，能否报告 expected/actual relation、候选规则和最小反例，并与原始 Lean 错误信息及普通 compiler log 比较。

图表计划：表 3: 基线能力对比；表 4: operator/relation coverage；表 5: 自动与手写工作量；表 6: mutation results；图 9: 编译时间与内存；图 10: 新模型迁移成本分解。

10 泛用性与剩余人工劳动

本节任务：直接回答“扩展到别的模型、训练框架、并行策略、world size 和硬件时要改什么”。

变化维度	已可复用	应当泛化但尚未完成	必须单独投入的工作
------	------	-----------	-----------

新模型, 同一 operator/layout	GraphDecl、store semantics、已有局部规则	graph-wide relation inference、certificate synthesis	确认模型输入条件与目标 observable
新 operator	图框架、proof composition	operator registry 与自动 side-condition plumbing	审计 Python 语义并证明新局部定理
新并行布局	authority、graph traversal	typed relation inference、generic adapter rules	定义布局重构语义并验证 runtime 实现
新训练框架/编译器	Lean core、关系库的一部分	通用 IR adapter interface	编写并审计新的 graph/provenance extractor
新 world size/topology	relation 抽象的一部分	从固定 cp=2 推广到参数化 rank set	collective scope、permutation 和边界情况证明
新硬件/kernel backend	数学 operator semantics	更通用的 authority capture	runtime selector、profile 与 kernel fidelity 审计
新输入协议	statement 与 witness 机制	contract schema 与自动 witness synthesis	给出真实 harness invariant 及独立检查

当前实现还包含 `num_dp=1`、`num_mb=1` 等范围限制, `bridge emitter` 也只 `dispatch` 已实现的 topology families。这些限制应进入表格和实验配置, 不能只藏在代码或附录。

明确划分三类:

1. **已经通用:** 图 fold、store、部分 tensor semantics、若干 relation、proof registry、kernel audit。
2. **设计上应通用但当前仍特化:** typed rule registry、全图 ownership inference、proof-certificate DAG synthesis、failure localization、参数化 world size。
3. **不可避免的人工工作:** 新算子或新布局的语义审计、训练框架 adapter、真实输入 contract、headline observable 的选择, 以及上游实现与形式语义的独立对照。

11 讨论与局限

本节任务: 解释抽象选择和未解决问题, 不把它们藏到结尾一句。

讨论 tensor semantics 与浮点/kernel fidelity 的边界; translator 尚未被形式化验证; native computation 的编译器信任; authority 供应链; 证明成本; false goal 与 counterexample extraction; world-size 参数化; 与训练框架维护者协作的责任分界。

12 相关工作

本节任务: 围绕问题关系组织文献, 不列关键词清单。

计划覆盖分布式 DNN compiler correctness、ML graph validation、translation validation、proof-producing compilation、interactive theorem proving for tensor programs、distributed semantics、runtime differential testing 和 artifact provenance。每个精确比较必须读取原论文后再写，当前提纲不预填优先权结论。

13 结论

本节任务：只重述有评估证据支撑的贡献，并给出下一步可证伪的泛化目标。

结论应落在“真实分布式图的值关系可以转化为内核检查的 proof obligation”上，同时承认当前 YOCO proof 仍有模型特化。下一阶段的明确验收是：在未用于开发的模型上，不改源码地生成完整 proof，或返回正确分类且可定位的失败。

A 计划附录

- A: 完整 paper-to-source map 与 theorem signature。
- B: operator denotation 和 relation rule registry。
- C: 五个 Goal 的 proof DAG、TID 和模块映射。
- D: authority、TCB、axiom baseline 与 sealed publication。
- E: 复现命令、mutation suite 和 artifact checklist。
- F: 历史错误归因与撤回记录。

成稿前必须补齐的证据

- nnScaler 从 llm-train compile API 到 SM/PM codegen 的实际时序与源码位置。
- SM/PM 代表性子图的可发表可视化，而不是直接截图数千节点。
- operator translation 的系统抽样与 mutation sensitivity。
- 自动生成与人工 proof 工作量的可复查统计。
- 相比 shape/numerical/compiler self-check 的公平基线。
- 至少一个 unseen architecture 的 clean-room 结果。
- 相关工作和贡献边界的主文献核验。
- 所有节点数、obligation 数、theorem 状态、测试结果和工件 digest 从同一 final receipt 机械生成，禁止拼接历史 manifest、coverage 文档和不同 revision 的数字。